

# Updating With Aggregation Pipelines

## Updating With An Aggregation Pipeline

- A pipeline is composed of multiple update expressions called stages.
- When an update operation containing multiple stages of update expressions is executed, each of the matched documents is processed and transformed through each stage sequentially.
- The output of the first stage is input for the next stage, until the last stage in the pipeline produces the final output.
- Using pipeline support, we can read and use the values of other fields in an update expression.
- If we wanted to use **updateMany()** with an aggregation pipeline update expression, we would have to specify an array of **multiple update expressions** (stages).
- **Description of example below:**
- **Stage 1 (\$set):** In this stage, we are using the **\$split operator** to split the full name with a white space. This gives us a two-element array containing the first name and the last name. We are also creating a **new field of name\_array** using the **\$set operator** and assigning the newly created array to it. **name\_array is a temporary field for us.**
- **Stage 2 (\$set):** In this stage, we refer to the array stored in name\_array and create new fields for the first name and last name. To do so, we use **\$arrayElemAt** on the name array to fetch its element from a specific index position. Two fields are created, **first\_name** and **last\_name**.
- **Stage 3 (\$project):** In the last stage, we project fields. We explicitly include the **first\_name** and **last\_name** fields and rewrite **full\_name** by concatenating **first\_name** in uppercase and leaving **last\_name** as it is. The **\$concat operator** accepts an array of strings and returns a single string by concatenating all the elements in the same order. The **\$project operator** is used to project fields and to assign them.

## Updating Array Fields

- It is possible to create / update array fields with the **\$set** operator.
- It is possible to remove array fields using the **\$unset** operator.
- It is clear that when an array field is being updated using the array as a value, it is treated like any other field.

### Adding Array Fields

- An operator called **\$push** can be used to add elements to the end of an array. If the array doesn't exist, it is created by the **\$push** operator.
- To add multiple elements to an array, we need to use **\$push** along with **\$each**.

### Sorting Arrays

- Arrays in MongoDB, and in general, are an ordered but unsorted collection of elements. The elements of the array will always remain in the order in which they were inserted.
- To sort an array, we need to use the **\$sort** operator with **\$each**. The **\$sort: 1** option sorts an array in ascending order (either numerically or alphabetically depending on the array's contents).
- The **\$sort: -1** option sorts an array in descending order (either numerically or alphabetically depending on the array's contents).
- It is also possible to sort fields of nested objects.

### Arrays As Sets

- An array is an ordered collection of elements that can be iterated over or accessed using its specific index position.
- A set is a collection of unique elements whose order is not guaranteed.
- MongoDB supports only arrays. It doesn't have support for sets. So, if we want to add only unique elements to an array, we can do so using the **\$addToSet** operator. This operator adds an element to an array only if it isn't present in it. The **\$each** operator also behaves like this when used with **\$push**.
- **Exercise 6.02 - New Category of Classic Movies -> Page - 291**

### Removing Array Elements

- The **\$pop operator**, when used in an update command, allows you to remove the first or last element in an array.
- It removes one element at a time and can only be used with the values **1 (for the last element)** or **-1 (for the first element)**:

### Removing All Elements

- If we want to remove all occurrences of a certain element from an array then we provide that element to the **\$pullAll operator** which removes all instances of it from the array.
- If we want to remove elements by matching them, then the **\$pull operator** can be provided with a **query** and all the elements matching that query will be removed.

### Updating Array Elements

- It is possible to use **\$[ ]** to update all the elements of an array to the same value.